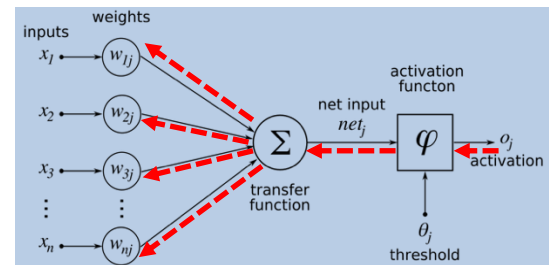


Backpropagation

Instructor: Hyunwoo J. Kim

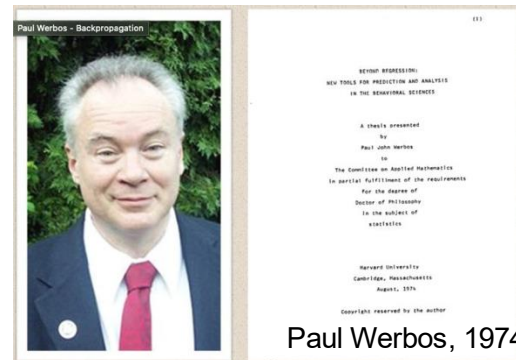
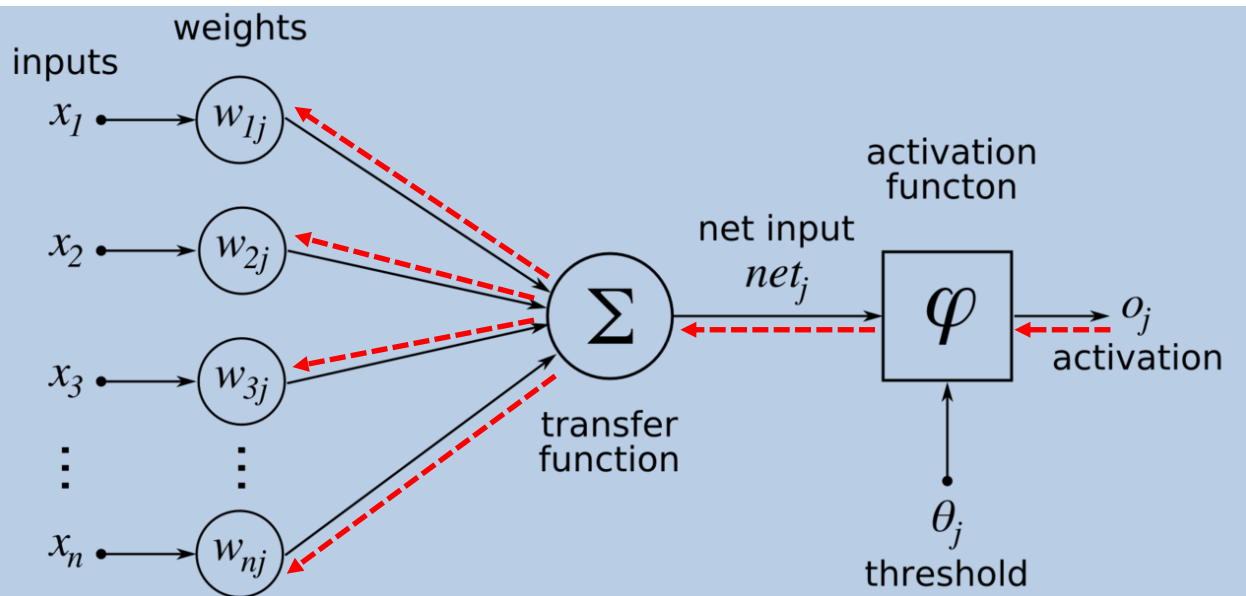


Agenda

- Backpropagation
 - Simple example
 - Simple neuron with sigmoid
 - Activation functions

Backpropagation

123



Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

*Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
†Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors². Learning becomes more interesting but

more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_{ij} , to unit j is a linear function of the outputs, y_i , of the units that are connected to j and of the weights, w_{ij} , on these connections

$$x_{ij} = \sum_i y_i w_{ij} \quad (1)$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights. A unit has a real-valued output, y_j , which is a non-linear function of its total input

$$y_j = \frac{1}{1 + \exp(-x_{ij})} \quad (2)$$

* To whom correspondence should be addressed.

Training!

- Define an ML problem (Task, Performance, Experience)
- Choose a model
- Choose a loss function (and regularizers)
- **Estimate the model (optimize parameters)**
- Test it!

Optimization? Gradients?

- Should we compute the gradient of the objective function (loss + regularization) for each variable by hand?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

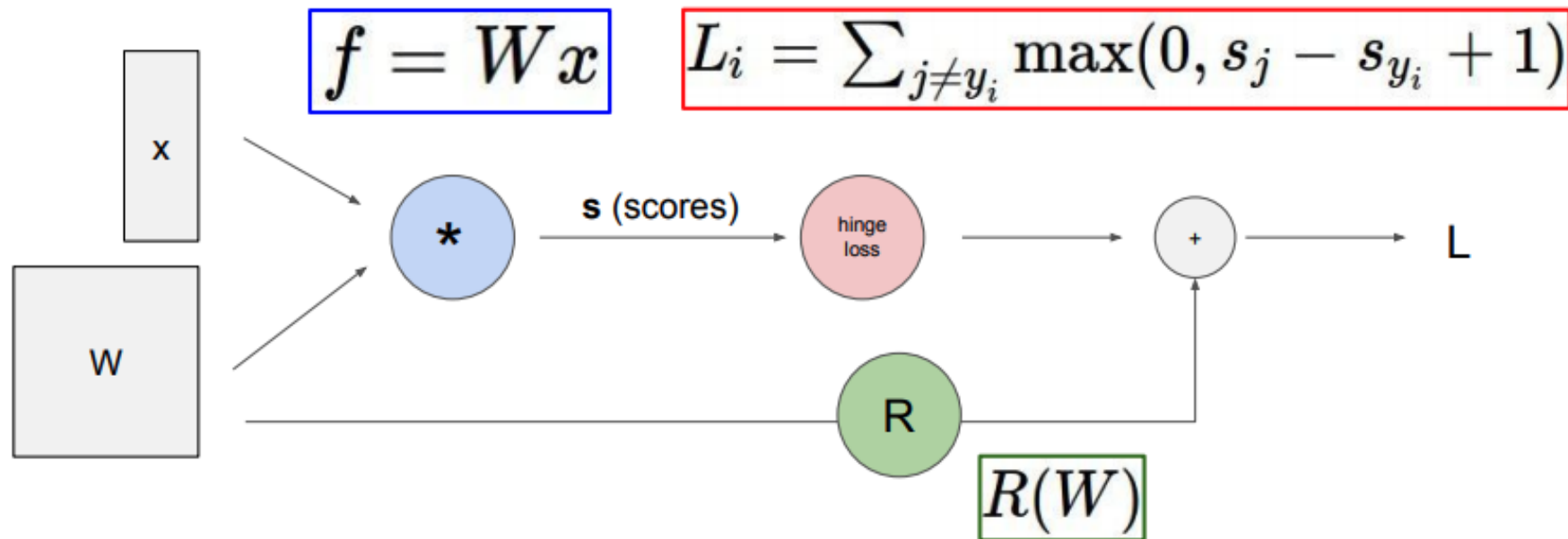
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM Loss on predictions}$$

$$R(W) = \sum_k W_k^2 \quad \text{Regularization}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad \text{Total loss: data loss + regularization}$$

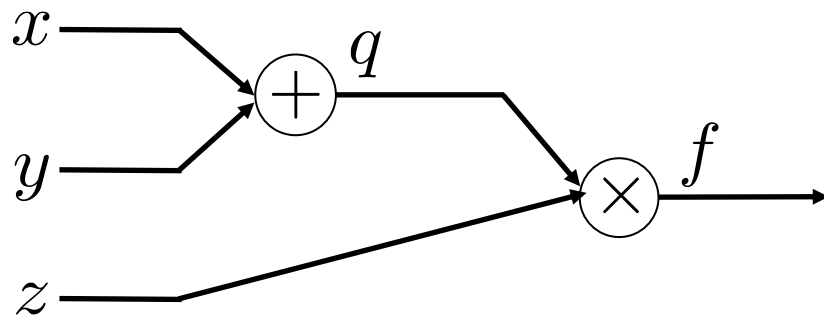
If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

Computational Graphs + Backpropagation



Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

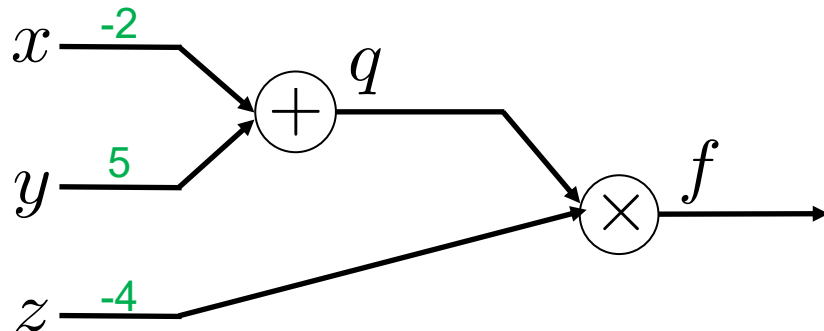


Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

Forward pass:

Input : $(x, y, z) = (-2, 5, -4)$

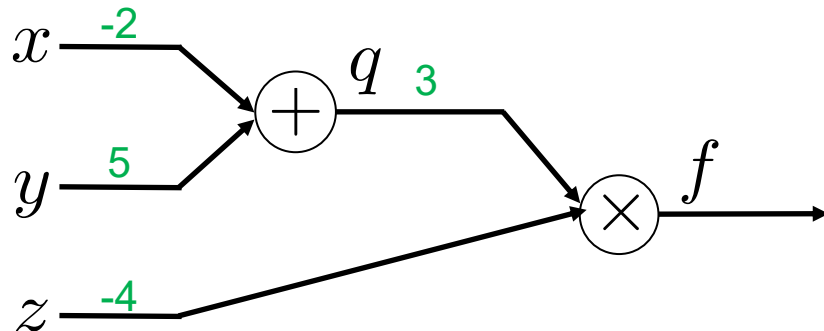


Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

Forward pass:

Input : $(x, y, z) = (-2, 5, -4)$

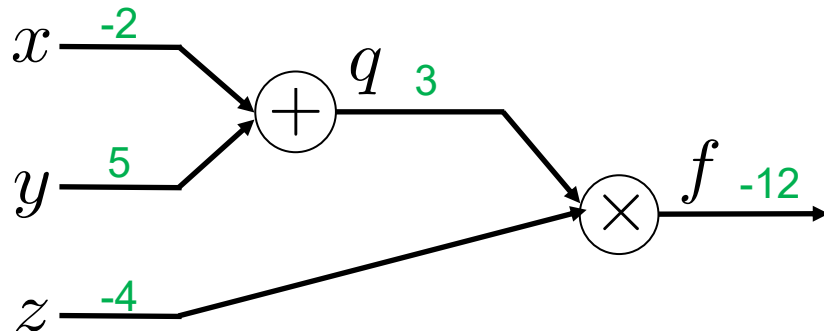


Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

Forward pass:

Input : $(x, y, z) = (-2, 5, -4)$



Backpropagation Simple Example

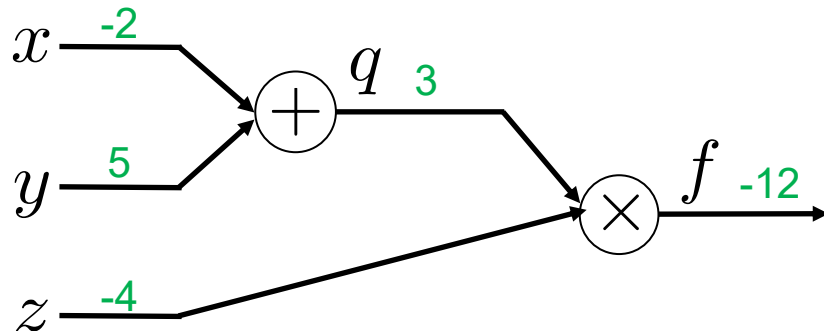
$$f(x, y, z) = (x + y)z$$

Forward pass:

Input : $(x, y, z) = (-2, 5, -4)$

How can we update x, y, z ?

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



Backpropagation Simple Example

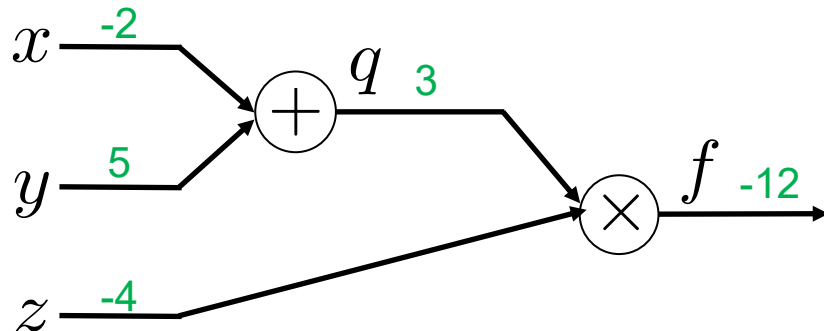
$$f(x, y, z) = (x + y)z$$

Forward pass:

Input : $(x, y, z) = (-2, 5, -4)$

How can we update x, y, z ?

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



What functions do we have?

$$q = x + y \quad \frac{\partial q}{\partial z} = 1, \frac{\partial q}{\partial y} = 1$$

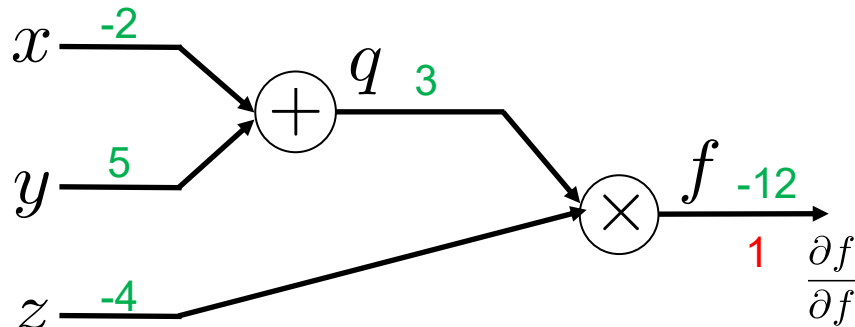
$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

Forward pass:

Input : $(x, y, z) = (-2, 5, -4)$



How can we update x, y, z ?

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

What functions do we have?

$$\begin{aligned} q &= x + y & \frac{\partial q}{\partial z} &= 1, \frac{\partial q}{\partial y} = 1 \\ f &= qz & \frac{\partial f}{\partial q} &= z, \frac{\partial f}{\partial z} = q \end{aligned}$$

Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

Forward pass:

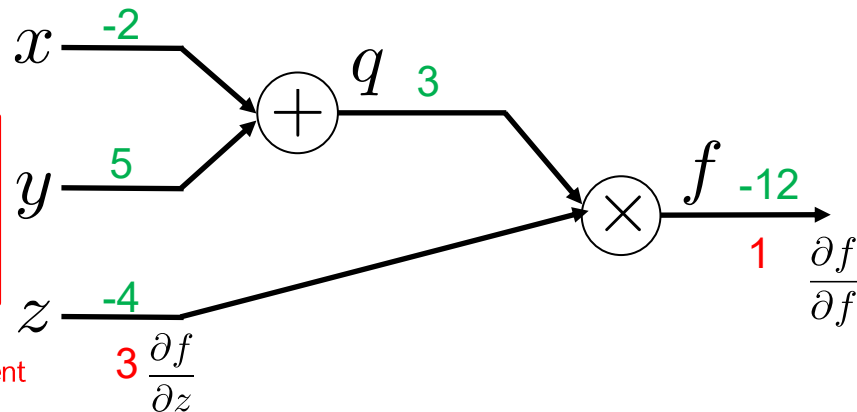
Input : $(x, y, z) = (-2, 5, -4)$

Chain Rule

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
Gradient

Local
Gradient



How can we update x, y, z ?

What functions do we have?

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

$$q = x + y \quad \frac{\partial q}{\partial z} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

Forward pass:

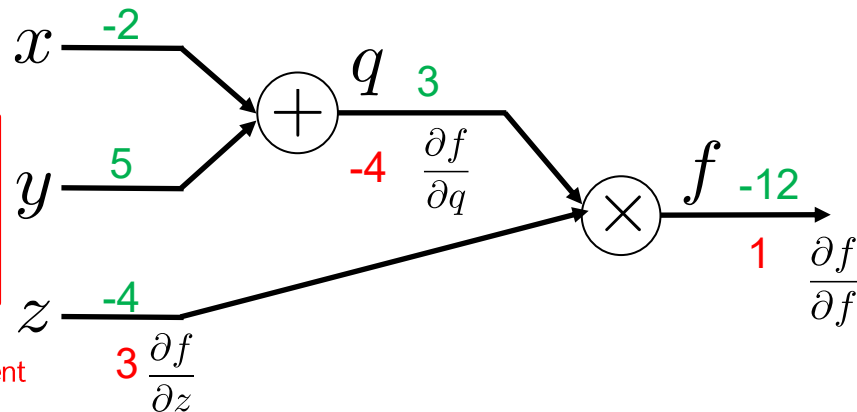
Input : $(x, y, z) = (-2, 5, -4)$

Chain Rule

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
Gradient

Local
Gradient



How can we update x, y, z ?

What functions do we have?

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

$$q = x + y \quad \frac{\partial q}{\partial z} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Backpropagation Simple Example

$$f(x, y, z) = (x + y)z$$

Forward pass:

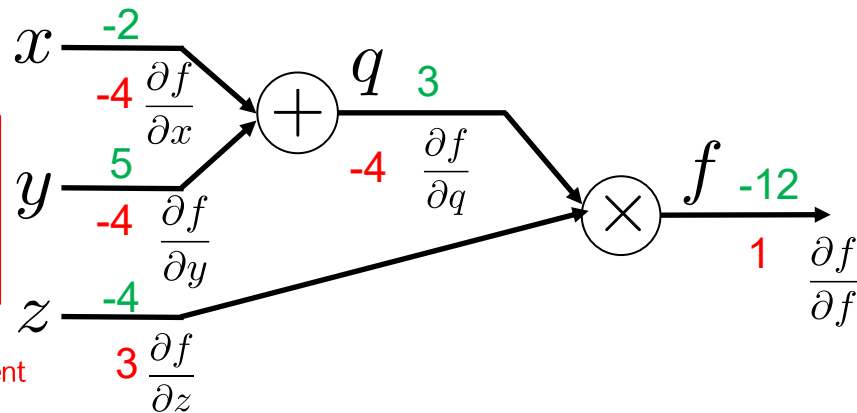
Input : $(x, y, z) = (-2, 5, -4)$

Chain Rule

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
Gradient

Local
Gradient



How can we update x, y, z ?

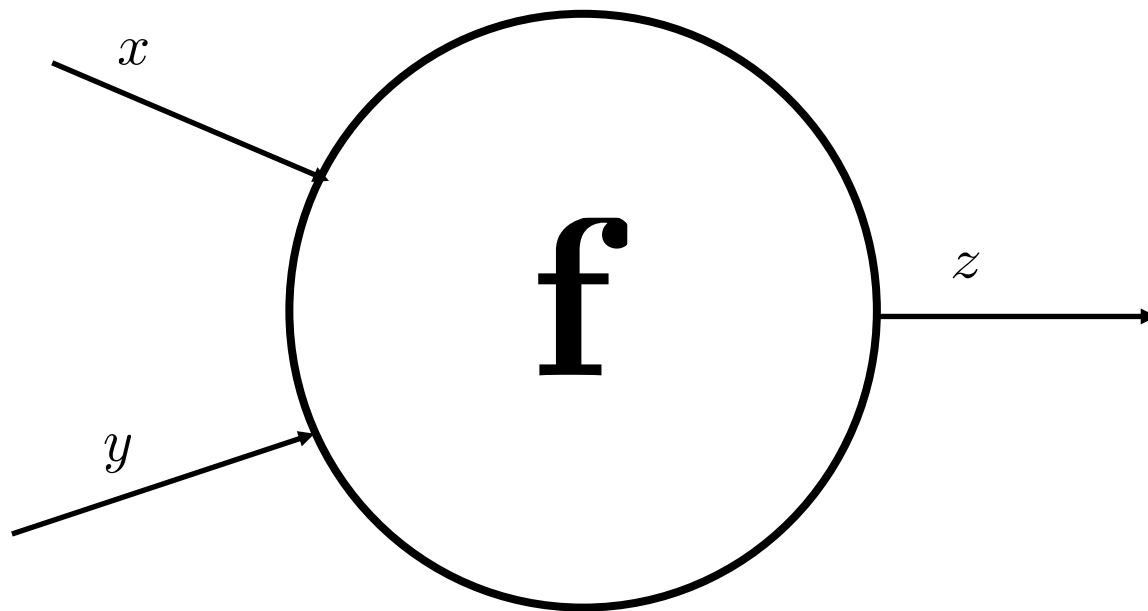
$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

What functions do we have?

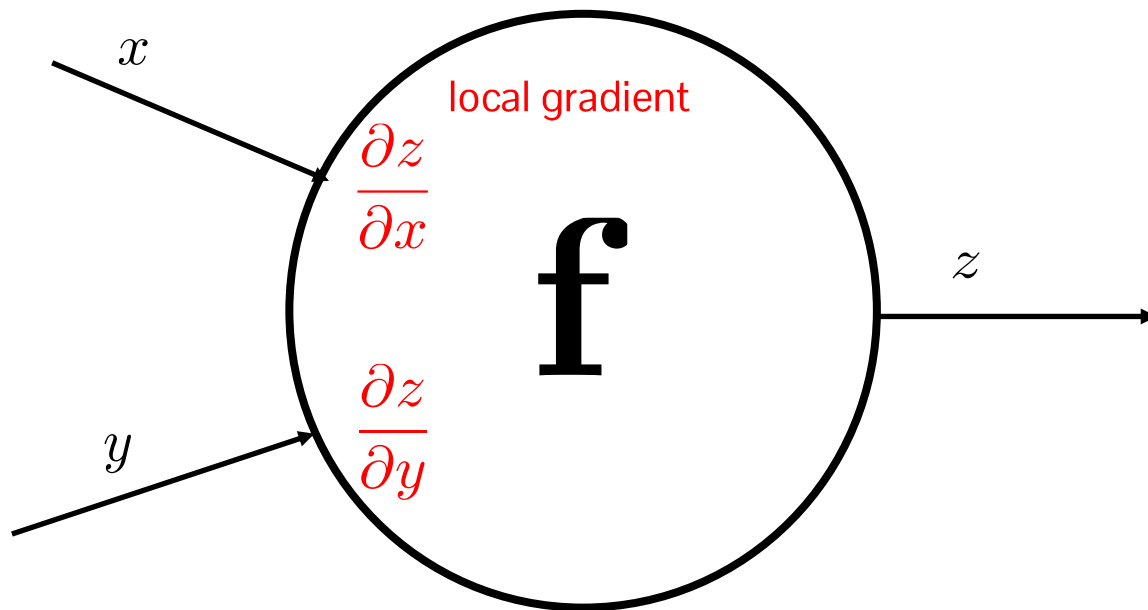
$$q = x + y \quad \frac{\partial q}{\partial z} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

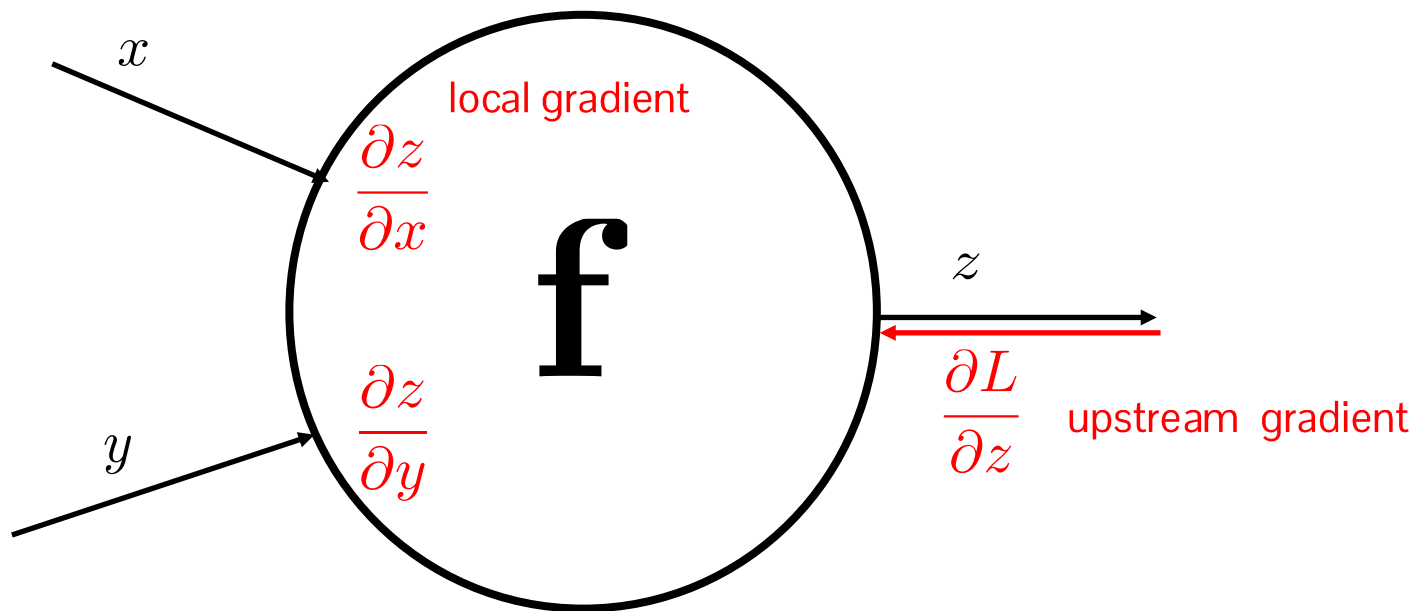
Upstream gradient & local gradient



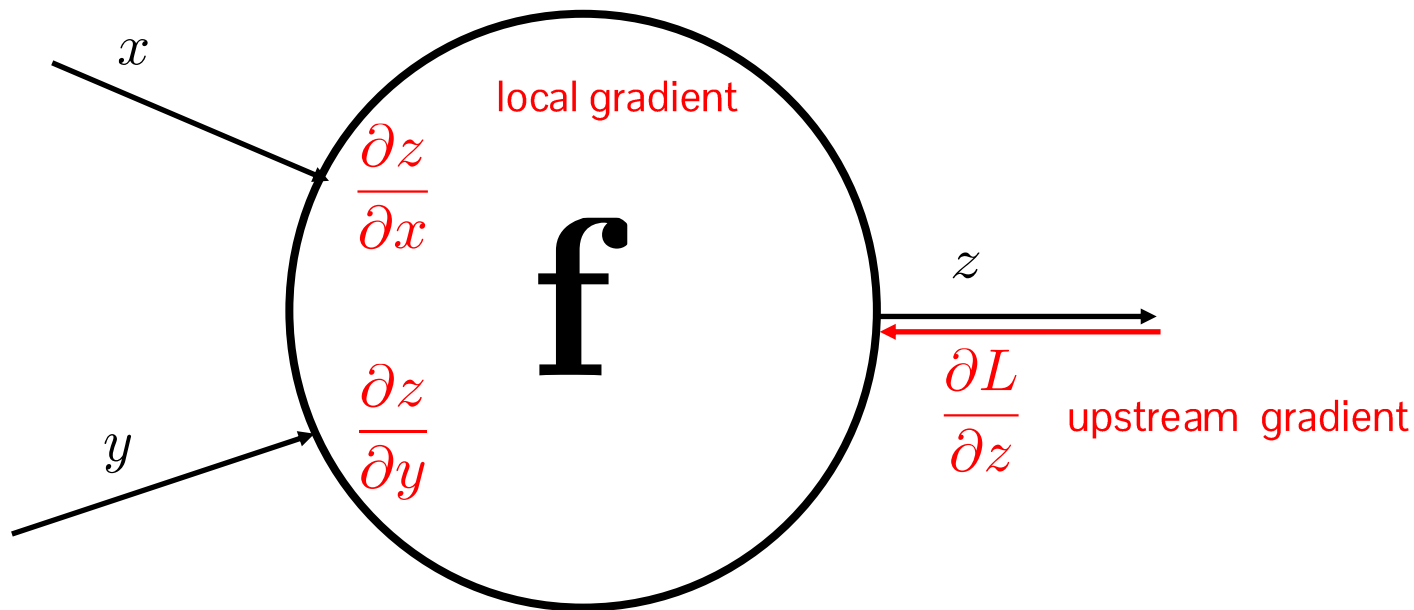
Upstream gradient & local gradient



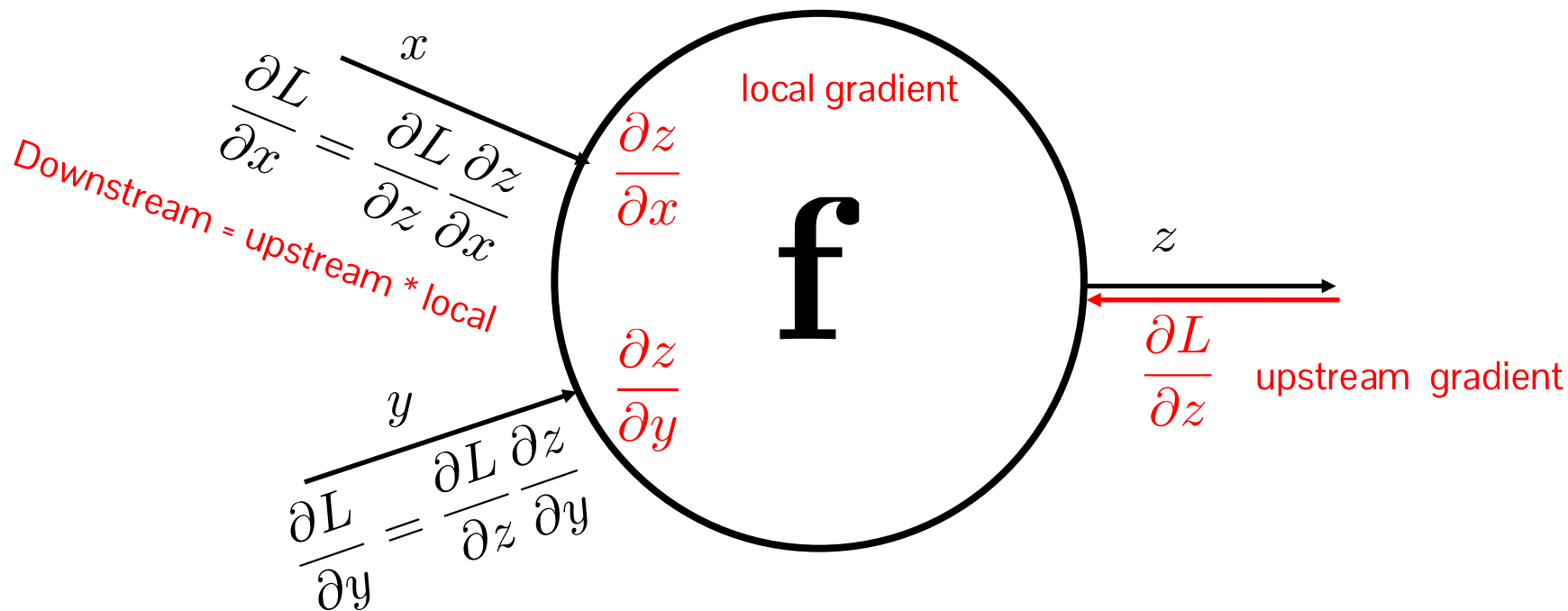
Upstream gradient & local gradient



Upstream gradient & local gradient

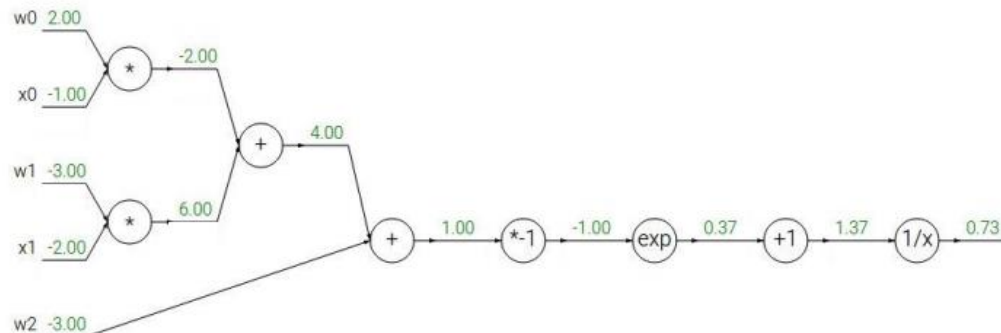


Upstream gradient & local gradient



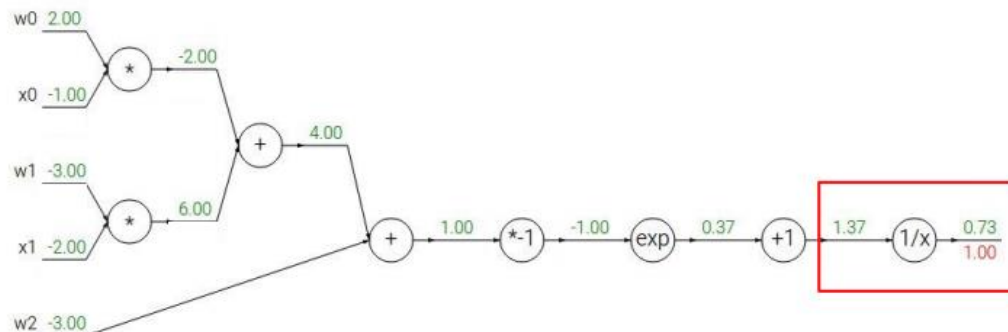
Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$$f(x) = e^x$$

 $\rightarrow$

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

 $\rightarrow$

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

 $\rightarrow$

$$\frac{df}{dx} = -1/x^2$$

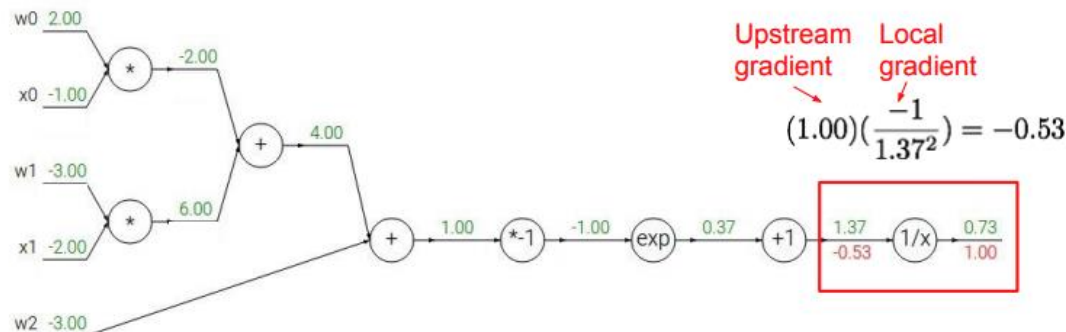
$$f_c(x) = c + x$$

 $\rightarrow$

$$\frac{df}{dx} = 1$$

Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$

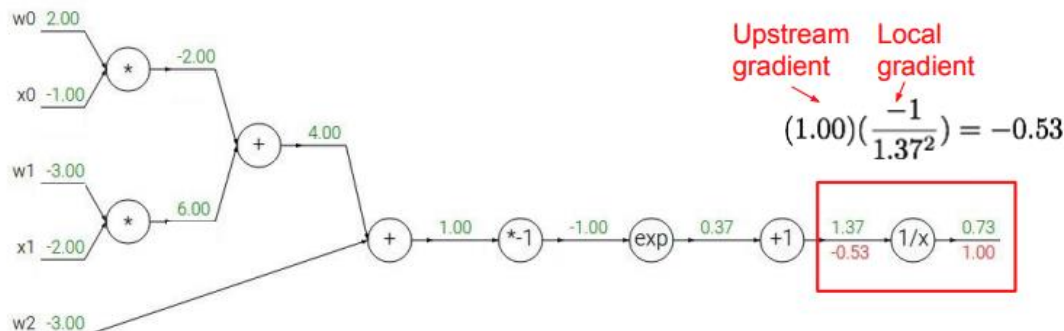


$$\begin{aligned} f(x) = e^x &\rightarrow \frac{df}{dx} = e^x \\ f_a(x) = ax &\rightarrow \frac{df}{dx} = a \end{aligned}$$

$$\begin{aligned} f(x) = \frac{1}{x} &\rightarrow \frac{df}{dx} = -1/x^2 \\ f_c(x) = c + x &\rightarrow \frac{df}{dx} = 1 \end{aligned}$$

Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$$f(x) = e^x$$

 $\rightarrow$

$$\frac{df}{dx} = e^x$$

$$f_a(x) = ax$$

 $\rightarrow$

$$\frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

 $\rightarrow$

$$\frac{df}{dx} = -1/x^2$$

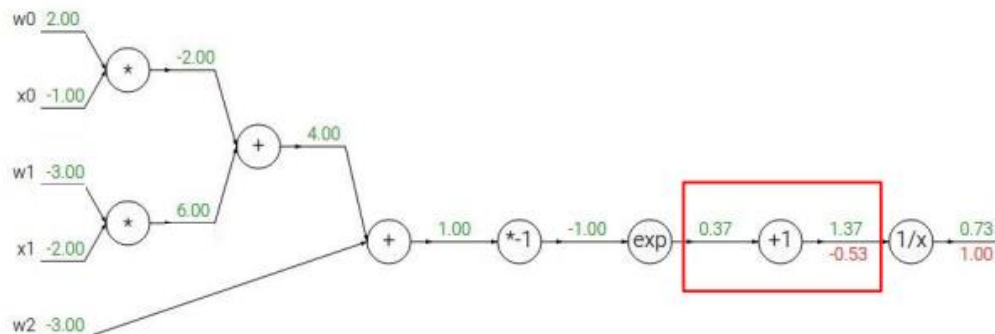
$$f_c(x) = c + x$$

 $\rightarrow$

$$\frac{df}{dx} = 1$$

Backpropagation for a simple neuron

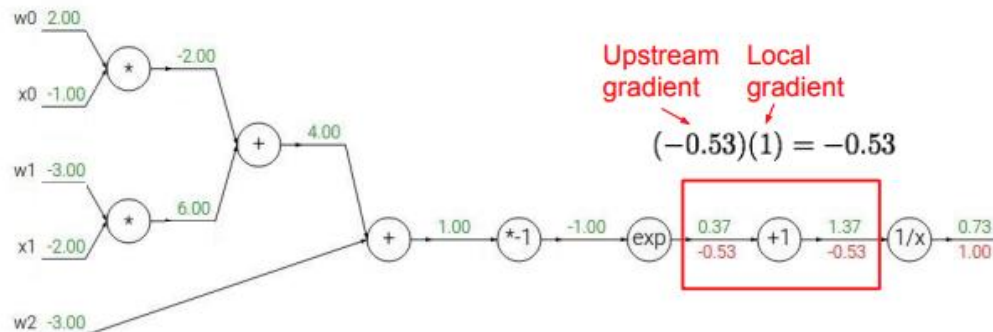
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Backpropagation for a simple neuron

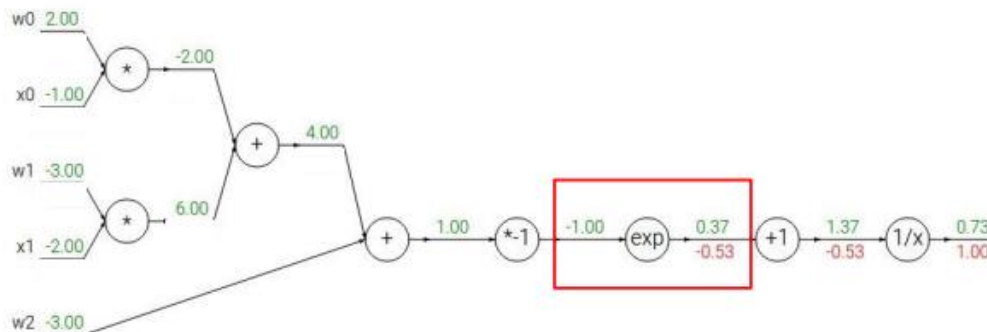
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Backpropagation for a simple neuron

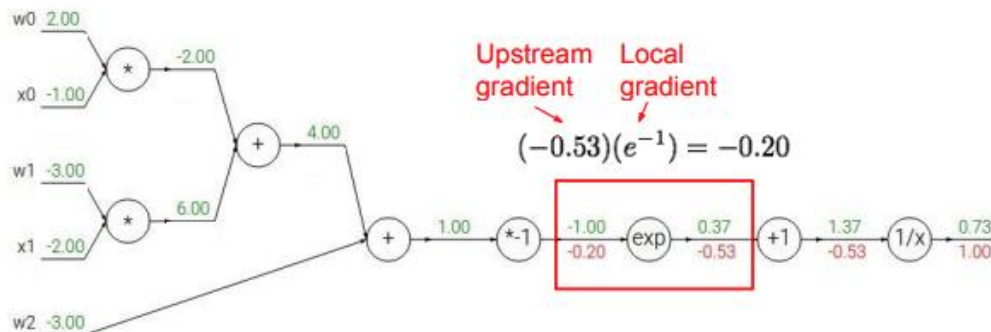
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

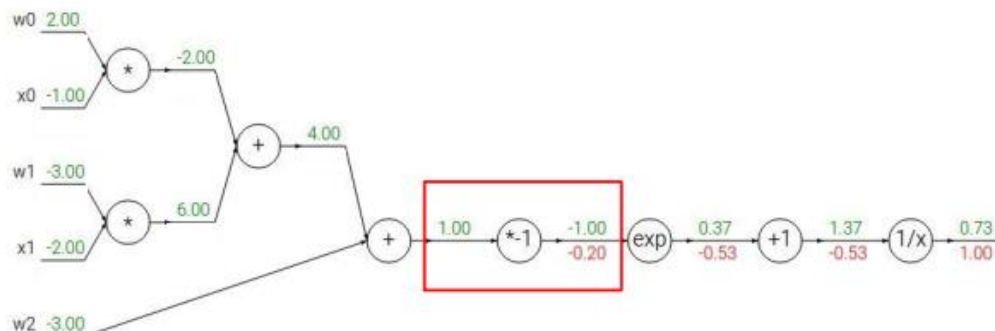
$$f_c(x) = c + x$$

$$\rightarrow \frac{df}{dx} = -1/x^2$$

$$\rightarrow \frac{df}{dx} = 1$$

Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

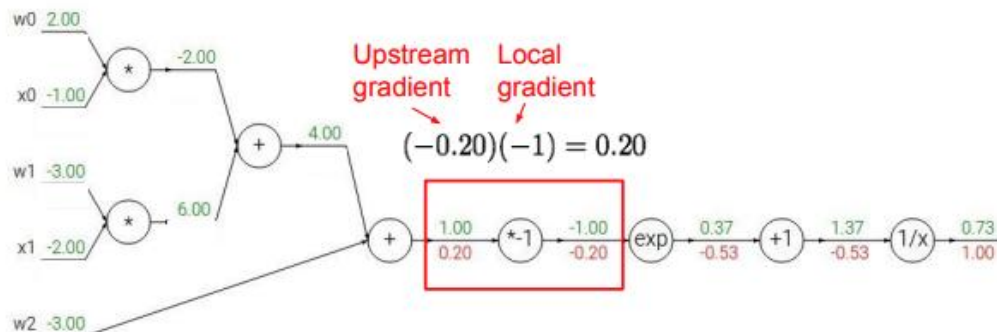
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

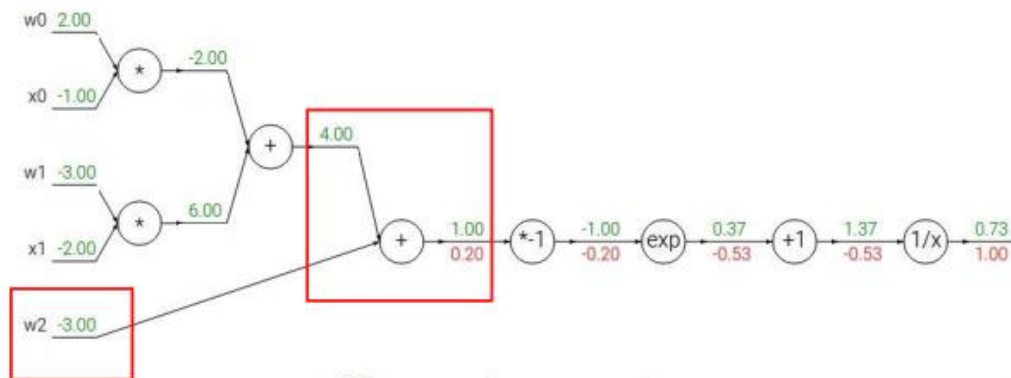
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation for a simple neuron

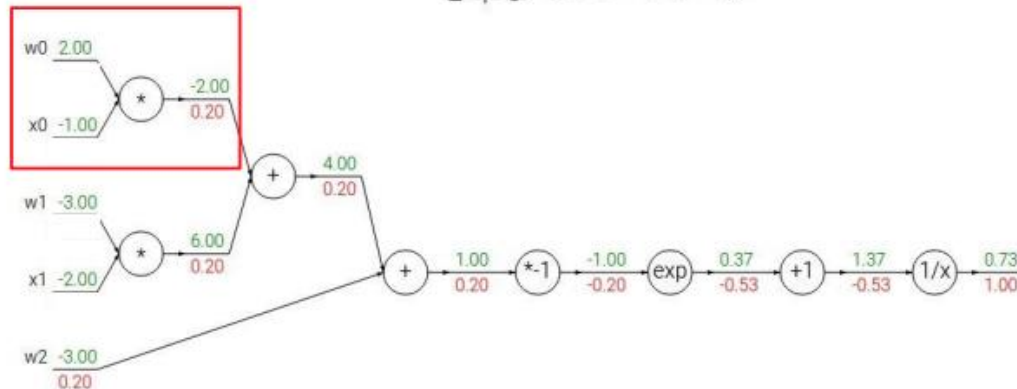
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Backpropagation for a simple neuron

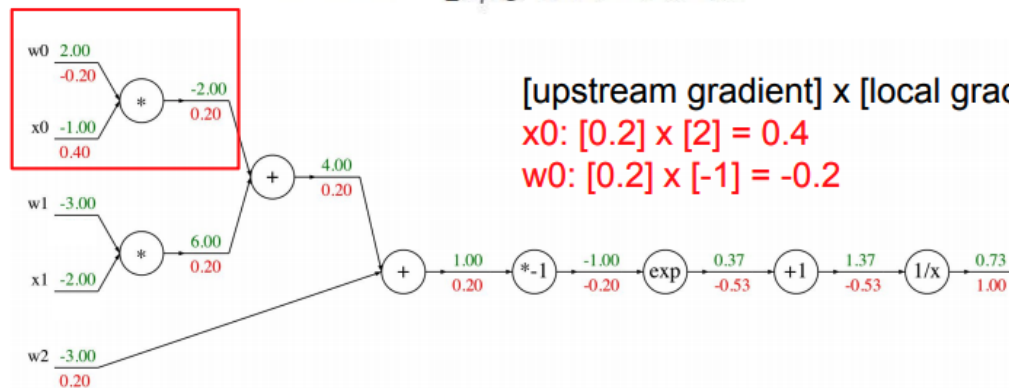
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$



$$\begin{array}{ll}
 f(x) = e^x & \rightarrow \frac{df}{dx} = e^x \\
 f_a(x) = ax & \rightarrow \frac{df}{dx} = a
 \end{array}
 \quad \Bigg| \quad
 \begin{array}{ll}
 f(x) = \frac{1}{x} & \rightarrow \frac{df}{dx} = -1/x^2 \\
 f_c(x) = c + x & \rightarrow \frac{df}{dx} = 1
 \end{array}$$

Backpropagation for a simple neuron

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



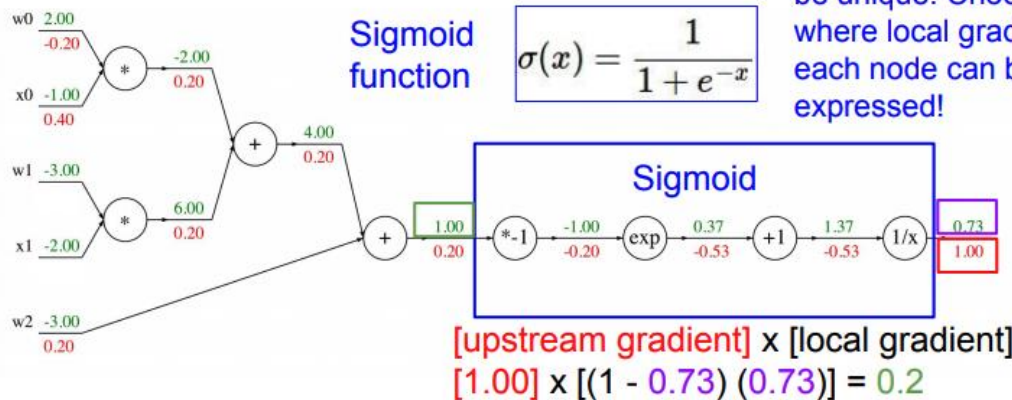
$$\begin{array}{ll}
 f(x) = e^x & \rightarrow \frac{df}{dx} = e^x \\
 f_a(x) = ax & \rightarrow \frac{df}{dx} = a
 \end{array}
 \quad \Bigg| \quad
 \begin{array}{ll}
 f(x) = \frac{1}{x} & \rightarrow \frac{df}{dx} = -1/x^2 \\
 f_c(x) = c + x & \rightarrow \frac{df}{dx} = 1
 \end{array}$$

Backpropagation for a simple neuron

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!

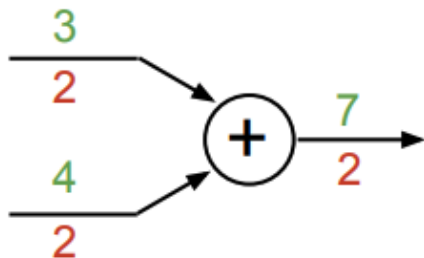


Sigmoid local gradient:

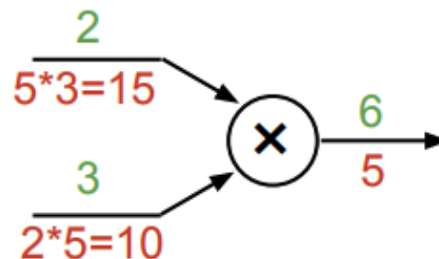
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

Patterns in gradient flow

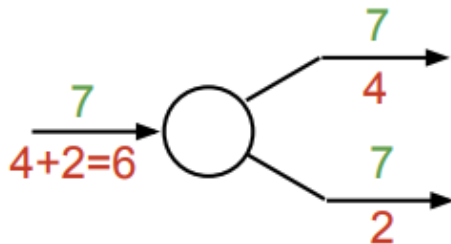
add gate: gradient distributor



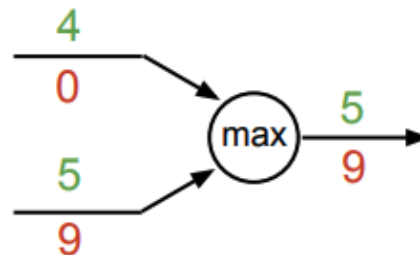
mul gate: “swap multiplier”



copy gate: gradient adder

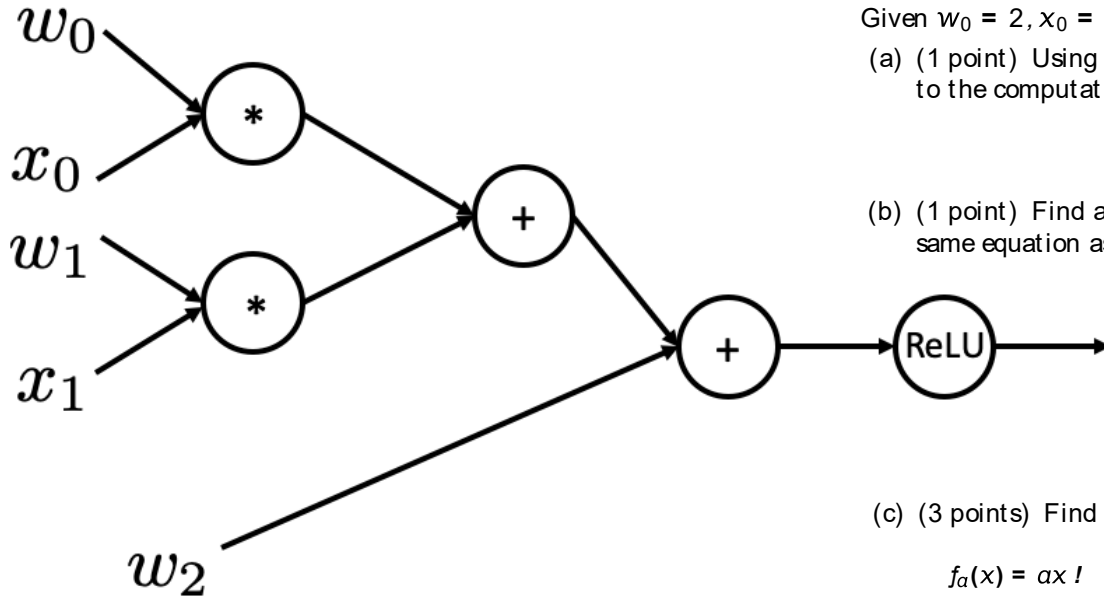


max gate: gradient router



Quiz

123



Given $w_0 = 2, x_0 = 0, w_1 = -1, x_1 = 1, w_2 = 2$, answer the following questions.

(a) (1 point) Using ReLU, $*$, $+$ and variables, write an equation that is equivalent to the computational graph in Fig. 2

(b) (1 point) Find a different computational graph that can be represented by the same equation as (a).

(c) (3 points) Find the first-order derivative of the functions below.

$$f_a(x) = ax \quad \frac{df}{dx} = \quad (1)$$

$$f_c(x) = c + x \quad \frac{df}{dx} = \quad (2)$$

$$f(x) = \text{ReLU}(x) = \max(0, x) \quad \frac{df}{dx} = \quad \text{if } x > 0 \quad (3)$$

$$f(x) = \text{ReLU}(x) = \max(0, x) \quad \frac{df}{dx} = \quad \text{if } x < 0 \quad (4)$$

(d) (2 points) Perform the forward pass. Write the all the intermediate values above the edges in Fig. 2.

(e) (3 points) Perform the backpropagation. Write the all the intermediate gradients below the edges in Fig. 2.

Questions

1. Why do we need the **chain rule** to train deep neural networks?
2. How does the choice of activation function affect gradient flow?
3. What causes the **vanishing and exploding gradient problems**?
4. How does batch size influence the quality of gradient estimates?

What's next?

- Scaling Laws!
 - How can we train a large model in a cost-effective way?